The Essential of Chapter 2

File organization and storage cut down to what the two mock exams ask: the page as the unit of cost, rows per page, heap against sorted against hashed, rows against columns, what `VACUUM` really does

Advanced Data Management

2026-10-29

How to read this document

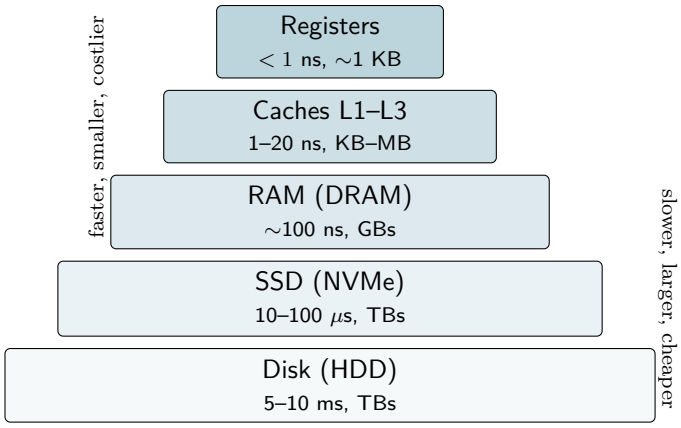
This is chapter 2 with everything removed that neither mock exam nor the chapter 1–3 refresher quiz touches. Five sections, each ending in a *Mock drill* box that says which mock question lives there and the shape of the answer the rubric wants. Read it in order: every term is defined before it is used. Numbers to carry: page 8 KB, page header 24 bytes, slot 4 bytes, PostgreSQL tuple header 24 bytes with 8-byte alignment; HDD random read 10 ms, SSD 100 μs, sequential 500 MB/s; `orders` 500,000 rows in 3,185 pages, `customers` 73 pages, `events` 50,000,000 rows in 1,000,000 pages.

1 The Page as the Unit of Cost

Concept: Block, page, and why you pay per page

A disk or SSD never reads one byte: it reads a whole **block** (typically 4 KB). The DBMS groups its data into fixed-size **pages**; PostgreSQL's are 8 KB. To read one row you read its entire page into memory, and the other rows on that page come along whether you wanted them or not. The real cost question is therefore never "how many bytes?" but **"how many distinct pages must I touch?"**

The reason the page count is a good clock is the memory hierarchy. Each level down is far larger and far slower, and the gaps are not small:



A CPU instruction takes about a nanosecond; a page read takes tens of microseconds on an SSD and about ten milliseconds on a spinning disk. One I/O is therefore worth 10^4 to 10^6 instructions, and for any operator that touches the disk the I/O term dwarfs the CPU term. Two more facts matter for the exam: **sequential** reads of adjacent pages stream at the device's bandwidth (500 MB/s in the mocks), while **random** reads pay a seek each (10 ms on an HDD, 100 μs on an SSD); and a page already in RAM costs nothing worth counting (chapter 6's buffer pool).

Key insight. Counting pages is within a few percent of the truth for anything that touches disk, it is hardware-independent, and it can be computed by hand from catalog statistics alone. The planner encodes the same judgement: `seq_page_cost` 1.0 against `cpu_tuple_cost` 0.01.

Mechanism: The I/O cost model, and how to turn pages into seconds

Let B be the number of pages a table occupies. The **cost** of an operation is the number of page reads and writes it performs; CPU work per page is ignored. A full scan costs B. To convert to time, multiply by the cost of one page on the device and access pattern: sequential, $B \times 8$ KB/500 MB/s $= B \times 16$ μs; random SSD, $B \times 100$ μs; random HDD, $B \times 10$ ms.

Table	B	sequential at 500 MB/s	random SSD	random HDD
`customers`	73	1.2 ms	7 ms	0.7 s
`orders`	3,185	51 ms (25 MB)	0.32 s	32 s
`events`	1,000,000	16 s (8 GB)	100 s	2.8 h

The model has three blind spots: it treats sequential and random reads as equal (the planner charges a random page four times a sequential one), it ignores whether a page is already cached (chapter 6), and it ignores operators that do heavy work per page:

Warning / Gotcha: When CPU dominates, and how to notice

The page count misleads you whenever an operator does a lot of work per page on data already in memory: a sort or hash aggregate over a table that fits in the buffer pool, a filter with a regular expression or `LIKE '%x%'` on a wide text column, field extraction from `jsonb`, an expensive function per row. The model says "B pages, cheap" and the clock disagrees. You

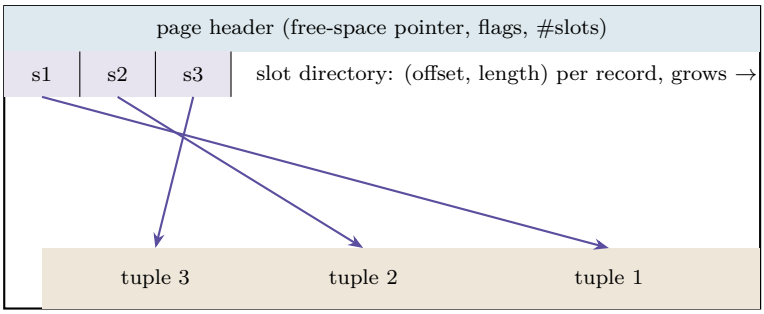
notice it in `EXPLAIN (ANALYZE, BUFFERS)`: few buffers, all `shared hit` and almost no `read`, yet a large actual time concentrated in a node that does no I/O (a `HashAggregate`, a `Sort` with `Sort Method: quicksort Memory:`, a `Seq Scan` whose `Filter` is a regex). Run the query twice: a page-bound query gets much faster once warm, a CPU-bound one does not.

Mock drill

Mock 1, 2.2 ("why we count pages, not instructions") is this section in three paragraphs: why pages (one I/O is 10^4–10^6 instructions; simple, hardware-independent, computable from B_R, $|R|$, M; the planner's own 1.0 against 0.01), one situation where CPU dominates, and how to notice it (`hit` high, `read` near zero, time unchanged when warm). **Mock 1, 3.2(a)**: 50,000,000/50 = 1,000,000 pages, ×8 KB = 8 GB, 8,000/500 = 16 s, the floor for any plan that reads `events` cold. **Mock v2 A7(c)**: `hit=40 read=3145` sums to 3,185, the whole of `orders`. Full chapter: §1–2, pages 1–4.

2 Inside a Page: Slots, Tuples and Rows per Page

A table is stored as one or more files; a file is a sequence of fixed-size pages; a page holds many records plus bookkeeping; a record holds the field values of one row. The engineering is inside the page. The naive layout, records packed one after another, has a fatal flaw: delete one in the middle and either you leave a hole or you shift every later record, changing its position; since indexes point at records by position, every delete would update every index. The slotted page fixes this with one indirection.



slots grow downward from the header; tuples grow upward from the bottom; free space sits between and shrinks from both ends

Mechanism: Slotted page

A **header** (24 bytes in PostgreSQL) records how many slots are in use and where free space begins and ends. A **slot directory** grows downward from the header; slot i holds the *offset* and length of record i within the page, 4 bytes per slot. **Tuples** grow upward from the bottom. **Free space** sits in the middle. A record is identified by **(page number, slot**

number), PostgreSQL's `ctid`, never by its byte offset.

Everything good about the layout follows from the indirection. **Stable row ids**: the slot holds the tuple's *current* offset, so the tuple can be moved within the page, to reclaim space after deletes, without changing its (page, slot) id and without touching any index. **Cheap delete**: deleting a row marks its slot free; the bytes are compacted later. **Variable-length rows**: each slot records offset and length, so records of different sizes coexist. A value too big for a page (a 2 MB document in a `text` column) is compressed and, if still too large, stored *out of line* in an overflow table with an 18-byte pointer left in the row: **TOAST**. The main table's scan stays fast however large those values are.

Mechanism: Rows per page, pages per table

Usable space is $8{,}192 - 24 = 8{,}168$ bytes. Each record costs its own size *plus* its 4-byte slot; in PostgreSQL the record size is the sum of the columns plus a 24-byte tuple header, rounded up to a multiple of 8. Then rows per page $= \lfloor 8{,}168/(\text{record}+4) \rfloor$, pages $= \lceil \text{rows}/\text{rows per page} \rceil$, size = pages $\times 8$ KB, scan cost = pages. The floor is not optional: records do not straddle pages.

Worked example: Three tables, by hand

Fixed 100-byte records, one million of them: $\lfloor 8{,}168/104 \rfloor = 78$ per page; $\lceil 10^6/78 \rceil = 12{,}821$ pages, about 100 MB. At 150 bytes: $\lfloor 8{,}168/154 \rfloor = \lfloor 53.04 \rfloor = 53$; forgetting the slot gives 54, forgetting the floor gives 53.04.

`tickets` (quiz C1): columns $8+4+12+8+8 = 40$ bytes, plus the 24-byte header = 64, already a multiple of 8; with the slot 68. $\lfloor 8{,}168/68 \rfloor = 120$ rows per page; $\lceil 3{,}000{,}000/120 \rceil = 25{,}000$ pages, 200 MB, a 0.4 s cold scan. Add a fixed 100-byte `notes` column: $140 + 24 = 164 \rightarrow 168$ aligned, 172 with the slot, 47 rows per page, 63,830 pages, 511 MB: every scan is 2.55× slower, even for queries that never read `notes`.

`orders`, read backwards: $500{,}000/3{,}185 \approx 157$ rows per page, about 52 bytes a row with its slot. Keep 157 for §5.

Mock drill

Quiz, chapter 2 block: the formula with header 24 and slot 4; the 53-record quick check; "why offsets in the slot directory" (tuples can move during compaction without changing the (page, slot) id that indexes store; variable-length tuples). **Quiz C1**: tuple size with header and alignment, rows per page, pages, MB, then the recomputation after a new column, floor and ceiling shown. **Mock 1, 3.2(c)** checks the arithmetic empirically with the queries of §5. Full chapter: §3, pages 4–7.

3 Heap, Sorted and Hashed Files

Given pages full of records, how are records arranged across the pages? A **heap** file puts a record wherever there is room. A **sorted** file keeps records in order of a search key across the pages. A **hashed** file sends a record to the page (**bucket**) $h(\text{key}) \bmod N$. We compare them in the only currency we have, pages touched, with B the number of pages.

Operation	Heap	Sorted	Hashed
Full scan	B	B	B
Equality search	B (avg $B/2$ if present)	$\log_2 B$	~ 1
Range search	B	$\log_2 B$+ matching pages	B (bad)
Insert	~ 1	up to B	~ 1

On a heap, finding one row by key means reading pages until it appears: $B/2$ on average when the key is present and unique (you can stop), but **always** B **when the key is absent**, because every page must be read to be sure. A sorted file allows binary search over pages, $\lceil \log_2 B \rceil$ reads, and a range query continues forward from the first match over contiguous pages; but an insert must land in key order, shifting half the file on average and all of it in the worst case. A hashed file computes the bucket and reads it, about 1 page; but adjacent keys hash to unrelated buckets, so a range query is a full scan.

Worked example: $B = 10{,}000$ **pages,** $\log_2 10{,}000 \approx 13.3 \to 14$

Operation	Heap	Sorted	Hashed
Find `id = 4275` (present)	5,000 avg	14	1
Find a key that is not there	10,000	14	1
All ids in 4000..4300 (5 pages of matches)	10,000	$14 + 5 = 19$	10,000
Insert one row	1	up to 10,000	1

At 10 ms a random read, the heap's 5,000 is 50 seconds to find one row; that is the pain an index removes. On the range row sorted wins by 500× and hashed is no better than heap; on the insert row sorted can be 10,000× worse. There is no single winner.

Heap favours inserts and scans; sorted favours reads by key, especially ranges; hashed favours exact match. The practical resolution, chapter 3, is to store the table as a heap and add *indexes*: separate structures that give sorted-file (log) or hashed-file (~ 1) lookup speed on top of the heap without the sorted file's insert price. PostgreSQL tables are heaps.

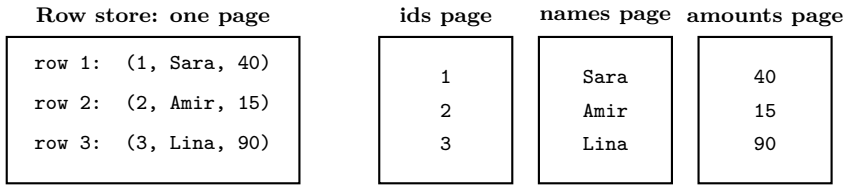
Mock drill

Neither mock asks this section directly, but the refresher quiz drilled it and the midterm's quick checks can. **Quiz quick checks**: an absent key costs B, not $B/2$; a sorted insert costs up to B; equality is $B/2$, $\log_2 B$, ~ 1. **Quiz C2**: choose per workload with the table: exact-match lookups

and batch loads, hashed; an append-only log read by full scans, heap; "all rows between two dates", sorted by date; a mixed workload, heap plus a B^+-tree ($h+1 \approx 4$ reads per lookup, inserts still cheap). Full chapter: §4, pages 7–9.

4 Rows Versus Columns

A **row store** (NSM, N-ary storage model) keeps all fields of a row together, so a page holds whole rows. A **column store** (DSM, decomposition storage model) keeps all values of one column together, so a page holds a slice of one column.



whole rows: "give me this record"

column store, one page per column: "aggregate this column"

Consider `SELECT avg(amount) FROM orders` on a table with 20 columns. In a row store every page holds whole rows, so to see every `amount` you must read *every page of the table*, dragging the other 19 columns through memory for nothing. In a column store the `amount` values sit on their own pages and you read only those: one twentieth of the data. Then compression: a column's values share one type and are often similar (sorted ids, repeated country codes, in-order timestamps), so run-length, dictionary and delta encoding shrink them far more than mixed-type rows. Less data read *times* better compression is why analytical engines (DuckDB, Parquet, warehouses) are column stores.

Why rows win for transactions: "update customer 42's address" is one record, one page read and one write in a row store; in a column store its fields are scattered over 20 pages, one per column, so the update becomes 20 reads and 20 writes. OLTP does single-record work all day; OLAP reads a few columns of many rows.

Worked example: Mock 1, 3.2(b): one column of `events`, measured

Counting events per day needs only `ts`, an 8-byte field. The PostgreSQL row store must read every page, 1,000,000 pages = 8 GB including the `jsonb` payload headers and every other column, to see one 8-byte field per row: 16 s cold at 500 MB/s. A Parquet file holding only `ts` is $50 \times 10^6 \times 8$ bytes = 400 MB raw, **20× less**, and in-order timestamps delta-encode to a few tens of megabytes, **50–100× fewer bytes**. DuckDB reads that one column vectorised and skips row groups by their min/max statistics. Export: `COPY (SELECT ts FROM events) TO '/data/ts.parquet' (FORMAT parquet)` with `pg_parquet`, or `FORMAT`

`csv` and let DuckDB convert; then `SELECT date_trunc('day', ts), count(*) FROM read_parquet(...) GROUP BY 1`. The syntax is a tool detail outside the chapter; the argument is the chapter.

Mock drill

Mock 1, 3.2(b) is the example above: claim (one column is read), number (8 GB against 400 MB raw against tens of MB compressed), mechanism (a page holds one column; same-type values compress; min/max skipping), then the two `COPY` forms. **Quiz quick checks**: 30 columns, one summed, the column store reads about 1/30 of the pages before compression; OLTP is whole-row reads and writes, row storage; OLAP is scans of few columns, column storage. Never write "column stores are faster": a single-row update is 20× slower. Full chapter: §5, pages 9–11.

5 PostgreSQL Storage and `VACUUM`

Everything above is visible in PostgreSQL. A table is one or more files of up to 1 GB; pages are 8 KB; rows live in a heap, in slotted pages, as a row store. Each tuple has a header (the transaction ids `t_xmin` and `t_xmax` that created and deleted it, flags), a null bitmap, then the field data; big attributes go to TOAST; `ctid` is the (page, slot) pair. Beside the table sit the **free-space map** (FSM), which records how much room each page has so an insert finds a page fast, and the **visibility map**, which marks pages whose rows are all visible to every transaction.

Interface / API: Seeing the storage yourself

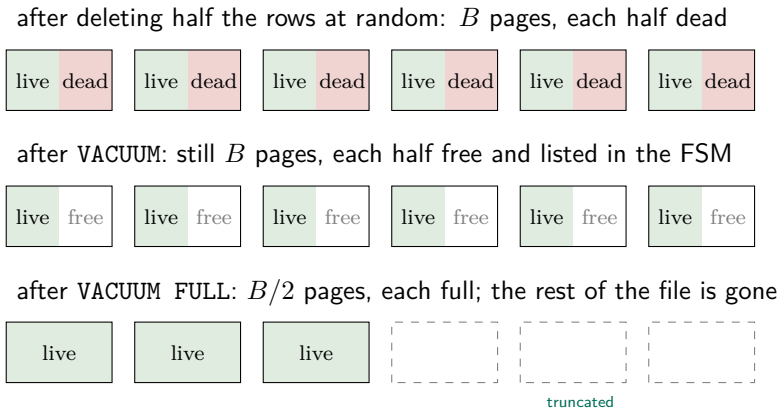
```
SELECT pg_relation_size('orders') / 8192 AS pages;              --
    3185
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('orders', 0));
   -- ~157
SELECT lp, lp_off, lp_len, t_xmax
FROM heap_page_items(get_raw_page('orders', 0)) LIMIT 2;
--  1 | 8152 | 40 | 0        2 | 8112 | 40 | 0
```

Read `heap_page_items` against the slotted-page figure: `lp` is the slot, `lp_off` the tuple's offset, decreasing as the slot number grows (8152, 8112): tuples really grow up from the bottom. A nonzero `t_xmax` marks a deleted tuple; a large gap between your estimate and `pg_relation_size` means dead tuples or TOAST.

Concept: Dead tuples, `VACUUM` and `VACUUM FULL`

`DELETE` does not remove bytes. It stamps the tuple's `t_xmax`; once no running transaction can still see the old version the tuple is **dead**, but it still occupies its bytes and its slot. A plain `VACUUM` removes dead tuples,

compacts each page's live tuples toward the bottom (updating their slots, so `ctid`s do not change), records the freed room in the free-space map, and truncates *trailing* pages that became completely empty. It **never moves a live row to another page**, so the file keeps its length unless its last pages are empty. `VACUUM FULL` (and `CLUSTER`) rewrites the whole table into a new file, packing live rows densely, at the price of an exclusive lock and a full copy.



Worked example: Mock v2 A2: delete half of `orders` at random, run a normal `VACUUM`

`orders` has 500,000 rows in 3,185 pages, 157 per page. A random delete of half the rows leaves about 78 live rows on *every* page: no page empties, so `VACUUM` has nothing to truncate. It frees the dead tuples' bytes inside each page and tells the FSM that every page is half empty, and the file stays at **about 3,185 pages**, each half full (answer b). The space is not lost: the next 250,000 inserts land in those holes and the file does not grow. Only `VACUUM FULL` or `CLUSTER` rewrites the table into $\lceil 250{,}000/157 \rceil \approx 1{,}592$ pages (answer a, the trap). The practical point: a `count(*)` still reads 3,185 pages after the plain `VACUUM`; deleting rows does not make a sequential scan cheaper until the table is rewritten.

5.1 Why the DBMS manages its own pages

The operating system already caches file pages in RAM, yet every serious DBMS keeps its own buffer pool (chapter 6). It knows three things the OS cannot. **The access pattern**: a large sequential scan touches each page once; the OS's generic least-recently-used cache would let it evict every useful page (index roots, hot customers), while the DBMS can drop the scan's pages first, pin pages in use and prefetch the next page. **The required write order**: durability needs the log record describing a change to reach disk *before* the data page it describes (write-ahead logging), and the log forced at commit; the OS writes dirty pages whenever it likes, in any order, so a crash between the two writes could leave a modified page on disk with no record of how to

undo it. **Its own memory budget**: a page cached once by the OS and once by the DBMS is **double buffering**; the DBMS sizes its pool deliberately and lets the OS cache cover the rest.

Mock drill

Mock v2 A2: answer (b), about 3,185, with the one-line reason "`VACUUM` frees space inside pages and truncates only empty trailing pages; random deletes empty no page", and name `VACUUM FULL` as what gives 1,592. **Mock 1, 3.2(c)**: `pg_relation_size / 8192` (expect 1,000,000) and `count(*)` over `heap_page_items` of page 0 (expect about 50), plus the sentence that the `jsonb` payload does not distort the count because it lives in TOAST. **Mock 1, 2.1** and **Mock v2 D3** draw their write-ordering paragraph from §5.1: say what breaks with the OS alone (a data page on disk before its log record, recovery impossible); chapter 6 supplies the steal/no-force and WAL detail. Full chapter: §6, pages 11–14.

Left out on purpose: the OS paging tour (§1.5), the "gap dramatised" table (§1.2), the fixed- versus variable-length record table (§3.4), hashed-file overflow and skew (§4.4), PAX (§5.3), the `generate_series` experiment and `iostat` (§6.1, §7), classic problems P6–P7; none is touched by either mock.

Warning / Gotcha: The four mistakes that cost points

- **Forgetting the slot or the floor.** Each record costs its size *plus* 4 bytes; 78.5 rows per page means 78; in PostgreSQL add the 24-byte header and align to 8 first.
- **"An absent key costs $B/2$."** No: $B/2$ is the average for a *present, unique* key. Proving absence reads every page, B.
- **"`VACUUM` shrinks the file."** A plain `VACUUM` frees space inside pages and truncates only empty trailing pages; with random deletes the page count does not move. Only `VACUUM FULL` or `CLUSTER` rewrites the table.
- **"Column stores are simply better."** They read 1/20 of the data for one column of twenty, and 20 pages instead of one for a single-row update; the workload decides.

Summary: The essential in five lines

1. Cost = pages touched: one I/O is worth 10^4–10^6 instructions. It lies only when heavy per-row CPU work runs on cached data (`hit` high, `read` near zero, time unchanged when warm). Pages to seconds: $\times 16$ μs sequential, $\times 100$ μs SSD, $\times 10$ ms HDD.
2. The slotted page gives stable (page, slot) ids; rows per page $= \lfloor 8{,}168/(\text{record} + 4) \rfloor$, record = columns +24 aligned to 8; `orders` holds 157 rows per page.
3. Heap: B to scan, $B/2$ for a present key, B for an absent one, ~ 1 to insert. Sorted: $\log_2 B$ and contiguous ranges, up to B to insert. Hashed: ~ 1 for equality, useless for ranges. Real systems: heap plus indexes.
4. Row store for OLTP, column store for OLAP: one column of twenty is $20\times$ less I/O before compression, 50–$100\times$ after; a single-row update costs one page against twenty.
5. `VACUUM` frees space inside pages and truncates only empty trailing pages (3,185 stays 3,185); `VACUUM FULL` rewrites to 1,592. The DBMS runs its own pool: it knows the access pattern, the write order, its memory budget.